

Assignment 3B Window Functions LLM Transcript – Brandon Chanderban

The following outlines the interactions I had with the LLM during the completion of this assignment, arranged in the order they occurred. Questions are denoted in green.

Q: How can I obtain financial data relating to the closing value of cryptocurrencies (namely Bitcoin and Ethereum)? Specifically, I want data spanning from the first of January 2022 to the 13th of February, 2026, to conduct some time series analysis.

One of the more straightforward ways to obtain cryptocurrency closing price data in R is through the **tidyquant** package, which acts as a tidyverse-friendly wrapper around financial data sources such as Yahoo Finance. After installing and loading tidyquant, the `tq_get()` function can be used to pull end-of-day price data directly into R.

For example, to retrieve daily closing prices for Bitcoin and Ethereum beginning January 1st, 2022, you may write:

```
library(tidyquant)
```

```
crypto_prices <- tq_get(  
  c("BTC-USD", "ETH-USD"),  
  from = "2022-01-01",  
  to   = "2026-02-13"  
)
```

This call fetches historical daily price data for both BTC and ETH in USD, returning a tidy dataframe containing variables such as symbol, date, open, high, low, close, and volume-related fields. From there, the dataset can be subsetting to focus specifically on the symbol, date, and close variables for subsequent time-series analysis.

Q: When dealing with R, how do you extract only the year from a date variable (in the format yyyy:mm:dd)?

When working in R, the most straightforward way to extract just the **year** from a date variable is by using the **lubridate** package, which is specifically designed for handling dates and times.

Assuming your date column is already stored as a proper Date object (for example, "2022-01-15"), you can extract the year as follows:

```
library(lubridate)
```

```
data_frame_with_year_variable <- data_frame%>%
```

```
  mutate(year = year(date))
```

The `year()` function pulls only the year component from the full date and stores it as a numeric variable.

If, however, your date column is still stored as a character string, you should first convert it to a Date format:

```
data_frame_with_year <- data_frame %>%
```

```
  mutate(date = as.Date(date),
```

```
    year = year(date))
```

Q: Could you please explain how to go about deriving the YTD average within a dataframe called `crypto_data`? It possesses columns titled: `symbol`, `date`, `close`, and `year`.

To derive the **Year-To-Date (YTD) average** within your `crypto_data` dataframe (which contains `symbol`, `date`, `close`, and `year`), you essentially need to compute a **cumulative average that resets at the start of each new year, and operates separately for each symbol**.

Since YTD means “from the beginning of the current year up to that specific date,” the process involves:

1. Grouping by both `symbol` and `year`
2. Ensuring the data is ordered chronologically
3. Computing a cumulative sum of `close`
4. Dividing by the number of observations up to that point

In `dplyr`, this can be done as follows:

```
crypto_data <- crypto_data %>%
```

```
  group_by(symbol, year) %>%
```

```
  arrange(date, .by_group = TRUE) %>%
```

```
  mutate(
```

```
    ytd_avg = cumsum(close) / row_number()
```

```
) %>%
```

```
ungroup()
```

Step-by-step explanation:

- `group_by(symbol, year)` ensures that BTC and ETH are handled independently, and that the calculation resets at the start of each new year.
- `arrange(date, .by_group = TRUE)` guarantees the observations are in chronological order within each group.
- `cumsum(close)` computes the cumulative total of closing prices within that symbol-year group.
- `row_number()` counts how many days have occurred so far in that year.
- Dividing the cumulative sum by the row number gives the cumulative (YTD) average.

The result is a new column `ytd_avg` that reflects the average closing price from January 1st of that year up to each specific date.

Q: How should I go about calculating the six-day moving average within the same `crypto_data` dataframe? And how should the first five days be dealt with (given that they do not comprise six days prior to them to compute the averages)?

To calculate the **six-day moving average** within the same `crypto_data` dataframe, you want a rolling average that looks at the **current day and the previous five days**, computed separately for each cryptocurrency.

Since this is a rolling window calculation, the `slider` package is particularly useful.

First, ensure the data is grouped properly and arranged chronologically:

```
library(slider)
```

```
crypto_data <- crypto_data %>%
```

```
  group_by(symbol) %>%
```

```
  arrange(date, .by_group = TRUE) %>%
```

```
  mutate(
```

```
    ma_6day = slide_dbl(
```

```
      close,
```

```
      mean,
```

```

    .before = 5,
    .complete = TRUE
  )
) %>%
ungroup()

```

```
head(crypto_data)
```

Explanation:

- `group_by(symbol)` ensures BTC and ETH are handled independently.
- `arrange(date, .by_group = TRUE)` guarantees the moving window follows chronological order.
- `slide_dbl()` applies a rolling function over the `close` column.
- `.before = 5` means: include the current observation plus the five previous days (total = 6 days).
- `mean` calculates the average within that rolling window.
- `.complete = TRUE` ensures that the function only computes a value once a full six-day window is available.

Regarding the first five days:

Since there are not enough prior observations to form a full six-day window, `.complete = TRUE` automatically assigns those first five observations a value of NA. This is standard practice in rolling calculations, as it avoids computing partial-window averages that would not truly represent a six-day period.

From day 6 onward, the moving average is computed normally as:

$$\text{Six-Day Moving Average} = \frac{\text{Sum of last 6 closing prices}}{6}$$

This produces a smoother short-term trend compared to the daily closing prices.

Q: Could you assist with a ggplot visualization? I want to have two time-series charts, side by side, one for each of the instruments (ETH and BTC). In these plots, I want to compare the actual closing values (within the `close` variable), to the computed six-day moving average (in the variable `mov_avg_6day`).

Yes — you can accomplish this cleanly using `ggplot2` with `facet_wrap()` so that BTC and ETH appear side by side, each in its own panel.

Here is the code:

```
library(ggplot2)
```

```
ggplot(crypto_data, aes(x = date)) +  
  geom_line(aes(y = close, color = "Closing Price"), alpha = 0.6) +  
  geom_line(aes(y = mov_avg_6day, color = "Six-Day Moving Avg."), size = 1) +  
  facet_wrap(~ symbol, scales = "free_y") +  
  labs(  
    title = "BTC and ETH Closing Prices vs Six-Day Moving Average",  
    x = "Date",  
    y = "Price (USD)",  
    color = "Legend"  
  ) +  
  theme_minimal() +  
  theme(plot.title = element_text(hjust = 0.5))
```

Explanation:

- `aes(x = date)` sets the time axis.
- The first `geom_line()` plots the actual daily closing price (`close`).
 - `alpha = 0.6` slightly fades it so the moving average stands out.
- The second `geom_line()` overlays the six-day moving average (`mov_avg_6day`).
 - `size = 1` makes it slightly thicker for visibility.
- `facet_wrap(~ symbol, scales = "free_y")` creates two panels:
 - One for BTC
 - One for ETH
 - `free_y` allows each cryptocurrency to use its own y-axis scale since their price magnitudes differ.
- `theme_minimal()` keeps the presentation clean.
- `element_text(hjust = 0.5)` centers the title.

The result will be two time-series plots placed side by side, clearly showing how the six-day moving average smooths out short-term fluctuations in the closing price for each cryptocurrency.